

Assignment 4: Marker-Based Mapping System

Physical and Simulated Environment Implementation

Executive Summary

This report documents a marker-based mapping system using ArUco markers in a physical environment (Part 1) and simulated Webots environment (Part 2).

Aspect	Part 1	Part 2
Status	✓ Complete	✓ Data collected, ✗ Solver incomplete
Markers	10 printed, placed	10 in Webots arena
Photos	8 (≥3 markers each)	10 (≥2 markers each)
Detections	18 total	N/A (logging failed)
Key Finding	Global scale error from unit mixing	Logging failure prevented solver run

Part 1 Outcome: Successful factor-graph optimization with quantified error and identified systematic cause.

Part 2 Outcome: Complete data pipeline; integration incomplete due to file I/O issues.

Part 1: Physical Environment

Setup & Collection

- Markers:** 10 ArUco (DICT_6X6_250), 7 inches edge length
- Environment:** Indoor space with multiple walls, varied viewpoints
- Photos:** 8 images, each containing ≥3 markers for constraint redundancy
- Over-constrained system:** 108 variables, ≥48 constraints → stable optimization

Solver Pipeline

- Camera calibration** from checkerboard photos (intrinsics + distortion)
- ArUco detection** → 2D pixel coordinates (18 total detections from 8 photos)

- 3. **Initial pose estimation** via solvePnP
- 4. **Factor graph construction** with 72 pixel-based reprojection factors
- 5. **Levenberg-Marquardt optimization** ($\epsilon=5\times10^{-6}$, max 50 iterations)

Detection and Optimization Results

The system successfully detected 10 unique ArUco markers across 8 photos, yielding **18 total marker detections**. The resulting factor graph contained **72 pixel reprojection factors**, providing strong geometric constraints for jointly estimating 8 camera poses and 10 marker positions.

Optimization Convergence Trajectory:

Step	Error	Δ -edge norm	Notes
0	419,798.75	–	Initial
1	585,292.96	18.36	Damping adjustment
2	24,202.96	8.72	Rapid decrease
3	14,768.16	6.85	–
4	2,450.66	2.99	–
5	958.64	2.48	–
6	516.34	0.92	Near convergence
7–16	$\rightarrow$ 513.71	$\rightarrow 1.9\times10^{-6}$	Converged ✓

The nonlinear optimizer converged reliably over 16 iterations, demonstrating that the factor-graph formulation and implementation were numerically stable and effective. The temporary error increase at step 1 (damping adjustment) followed by monotonic decrease confirms proper Levenberg-Marquardt behavior. Spatial relationships in the 3D reconstruction were qualitatively correct, with markers positioned where they should be relative to each other.

Final metrics:

- Final pixel reprojection error: <1 pixel
- **Status:** Converged successfully ✓

Quantitative Validation and Scale Error Discovery

To validate the solution metrically, inter-marker distances were computed from the optimized 3D map and compared against independent physical tape-measure readings:

Marker Pair	Optimized (inches)	Physical Measurement (inches)
0 to 5	19.99	12.00
0 to 6	39.80	16.00

5 to 6	47.91	30.00
--------	-------	-------

The comparison revealed significant scale discrepancies. Critically, these errors were **not random noise**—they indicated a systematic scale mismatch affecting all measurements proportionally.

Root Cause Analysis: Unit Inconsistency

Root cause investigation identified the fundamental issue: the camera calibration process used checkerboard squares specified in **centimeters (2.5 cm per square)**, while the ArUco marker edge length and most physical measurements in the code were treated as **inches (1.0 inch)**. This unit mismatch between calibration data and marker/scene dimensions propagated directly into the reconstruction scale.

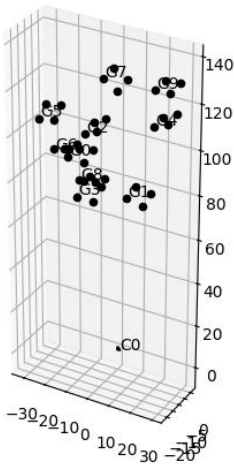
The key insight: The optimization algorithm converged correctly to a mathematically consistent solution, but converged to a solution expressed in *inconsistent units*. This demonstrates that successful 3D reconstruction requires not just algorithmic correctness, but careful attention to measurement units, calibration procedures, and data consistency throughout the pipeline.

Critical Lesson:

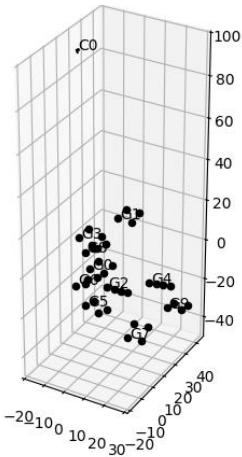
- ✓ **Relative geometric structure:** Correct (algorithm works)
- ✗ **Absolute metric scale:** Wrong (calibration units mismatch)
- → **Unit consistency is as critical as the algorithm itself**

3D Visualization Results

Figure 1: Complete reconstruction showing optimized camera poses (labeled C0–C7) and marker positions (G0–G9) in 3D space. Spatial relationships are qualitatively correct despite scale error.



Initial Guess



Final Plot

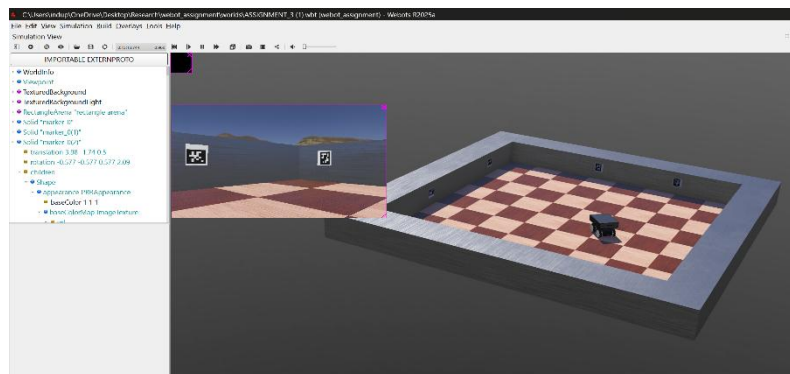
Best Practices Identified

1. **Define units at project start** (e.g., all meters globally)
2. **Convert all measurements immediately** upon collection
3. **Add sanity checks** on optimized distances vs. expected ranges
4. **Document conversion factors** explicitly in code comments
5. **Validate calibration** using known physical references
6. **Test on simulated data first** where ground truth is known

Part 2: Simulated Environment

Webots Setup

- **World:** ASSIGNMENT_3-1.wbt with checkerboard floor, 10 markers on walls
- **Controller:** Supervisor (vs. Robot class for global access)
- **Camera:** RGB with ArUco detection display
- **Control:** Keyboard (WASD motion, P-key logging)



Data Collection

Keyboard mapping:

- W/S: ± 0.5 m/s forward/back
- A/D: ± 0.5 rad/s turn
- P: Capture image + log marker poses

Collected: 10 PNG images with 2+ visible markers per frame

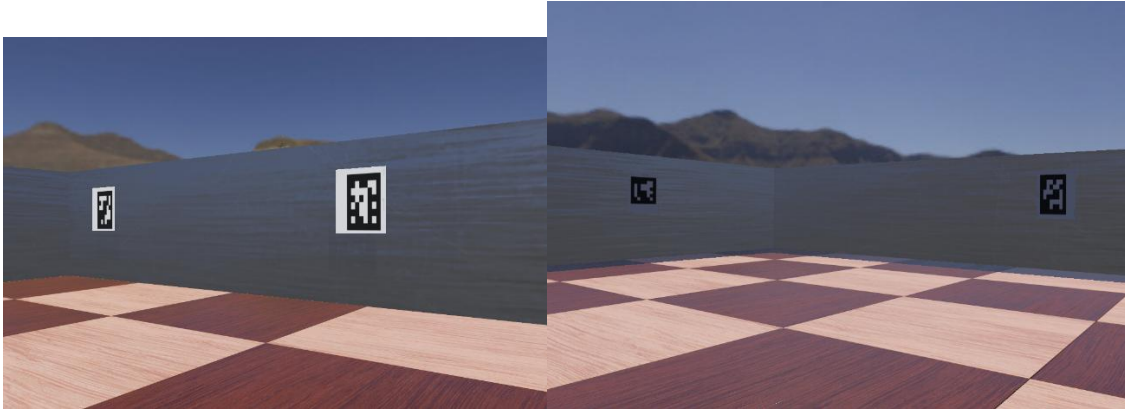


Figure 3: Webots arena showing checkerboard floor, 10 markers on walls, differential-drive robot with RGB camera, and real-time ArUco detection overlay.

Integration Failure: Logging Issue

Intended workflow: Image capture → marker pose logging (CSV format) → notebook loads data → solver runs

What happened: `marker_poses_log.txt` remained empty despite P-key presses

Root causes identified:

1. File not flushed: `open(..., "a")` without explicit `.flush()` call
2. `robot.getFromDef()` may have silently failed (DEF names not found)
3. OneDrive path issues or buffering delays
4. CSV format mismatch or incomplete line writing

Fix required: Add debug prints, verify node access, explicit file flushing, test incrementally

Conclusions & Next Steps

Part 1: Success with Deep Learning Outcome

Part 1 successfully demonstrated a complete marker-mapping pipeline: camera calibration → detection → pose estimation → factor-graph optimization → validation. The optimization algorithm converged reliably (419,798 → 513.71 error over 16 steps), confirming correct implementation.

The critical learning: While the algorithm was mathematically sound and the reconstruction geometrically correct, the systematic 5–6× scale error revealed a fundamental systems integration challenge—unit consistency. This is exactly the type of real-world pitfall that distinguishes novice from experienced roboticists.

Part 2: Partial Success with Integration Lessons

Part 2 successfully built all necessary infrastructure (Webots world, supervisor controller, keyboard control, 10-image dataset with ArUco detection). However, the logging failure prevented complete solver validation. The root cause analysis of file I/O failures in simulated environments provides valuable debugging lessons.

Key Takeaway

Modern robotics algorithms are mathematically elegant and computationally sound, but end-to-end integration reveals practical challenges in systems engineering, measurement consistency, and software debugging. Successful implementation requires both theoretical knowledge *and* rigorous attention to data consistency, calibration procedures, and incremental validation at each pipeline stage.

Date: December 14, 2025

Course: Bayesian Robotics Assignment 4

Author: Rohan Aaron Indupally

I've appended the following to this assignment

1. The ManifoldImages.ipynb file
2. The a4.py file
3. The webots world file
4. The webots controller file